AI in Production

Observability, Traceability, Monitoring, Evals, Deployment and Design

A 150-Question Interview Bank with Answers

Observability 20 · Traceability 20 · Monitoring 20 · Evals 20 · Deployment 45 · Math and Design 25

Covering GenAI, Agents, MCP, model gateways, OpenTelemetry GenAI conventions, RAGAS, attention math, and system design

How to Use This Guide

This bank targets engineers preparing for applied-AI, ML-platform, AI-infra, and MLOps roles where the hard questions are no longer whether you can build a RAG system, but how you know it works once it is live and how you keep it working. Each question carries a difficulty badge and a company-style tag.

EASY	Foundational. Expected of all candidates.
MEDIUM	Design and depth. Show you understand the trade-offs.
HARD	System design, production failure modes, and scale. Senior or Staff level.

A note on the company tags: they indicate the **archetype and style** of company that typically probes this kind of question, whether a big-tech systems shop (Meta, Amazon, Google, Microsoft, Netflix), a product or SaaS company (Atlassian, Stripe, Notion, Uber), an AI-infra or observability vendor (Datadog, Databricks, Arize, LangChain, Anthropic, OpenAI, Nvidia), or a standards and tooling body (RAGAS, DeepEval, Okta, Render). They are representative interview contexts to help you calibrate depth and framing, not verified questions from any firm.

Section 1: GenAI and Agent Observability

20 questions on the metrics, logs, and traces that reveal what an LLM or agent system is doing internally: OpenTelemetry GenAI semantic conventions, token, cost, and latency signals, RAG-specific telemetry, content-capture trade-offs, multi-agent span trees, and vendor-neutral instrumentation.

Q1. EASY DATADOG

What are the three pillars of observability, and how do they map to LLM applications?

Answer

The pillars are metrics, logs, and traces. For LLM apps, metrics cover token usage, latency (TTFT, tokens per second), cost, and error rate; logs cover prompts, completions, and tool inputs and outputs; traces cover the end-to-end request path across LLM calls, retrieval, and tool calls. Observability is the ability to infer internal system state purely from these external signals, including for failures you did not anticipate.

Q2. EASY OPENAI

Which signals should you capture for every single LLM API call?

Answer

Model name, input and output token counts, total latency, time-to-first-token, finish_reason, derived cost (tokens times price), and status or error. These map directly to the OpenTelemetry GenAI metrics gen_ai.client.operation.duration (a latency histogram) and gen_ai.client.token.usage (a token histogram), plus the gen_ai.request.model attribute.

Q3. EASY MICROSOFT

What is the difference between observability and monitoring?

Answer

Monitoring watches known metrics against thresholds and alerts on failures you predicted. Observability is the broader ability to ask arbitrary new questions about system behaviour from rich telemetry, which lets you debug unknown-unknowns. Monitoring tells you that something is wrong; observability tells you why.

Q4. MEDIUM DATADOG

Walk through the OpenTelemetry GenAI semantic conventions. Which span types and attributes matter?

Answer

The gen_ai.* namespace is backed by CNCF and adopted by Datadog, AWS, Azure, and Google. Span operations include chat (an LLM call), execute_tool, invoke_agent, and embeddings. Key attributes are gen_ai.request.model, gen_ai.response.model, gen_ai.usage.input_tokens, gen_ai.usage.output_tokens, gen_ai.response.finish_reasons, and, when content capture is on, gen_ai.system_instructions, gen_ai.input.messages, and gen_ai.output.messages. Metrics are gen_ai.client.operation.duration and gen_ai.client.token.usage. Being vendor-neutral means telemetry is not locked to a proprietary format.

Q5. MEDIUM LANGCHAIN

How do you instrument a multi-step agent for observability?

Answer

Wrap the whole run in a root invoke_agent span. Each LLM call becomes a child chat span, each tool call an execute_tool span, and each retrieval a retrieval span. Capture per step the model, tokens, latency, tool name, arguments, results, and the decision made. Propagate trace context so all spans share one trace_id, and export through an OTel exporter to a backend such as Phoenix, Langfuse, or Datadog.

Q6. MEDIUM UBER

Distinguish a session, a trace, and a span in conversational AI observability.

Answer

A span is one unit of work, such as a single LLM or tool call. A trace is one full request, for example a user turn that includes retrieval, generation, and tools. A session is an entire conversation spanning many turns and traces, tied together by a session or thread id. You need all three: the span to find the failing call, the trace for the turn, and the session for multi-turn context issues.

Q7. MEDIUM RAMP

Why is capturing prompt and completion content in spans both valuable and risky?

Answer

It is valuable because it is essential for debugging quality problems and for replay. It is risky because content can be very large, which bloats storage and cost, and it often contains PII or secrets, which is a privacy and compliance exposure. Mitigate with sampling, PII redaction before export, content-capture toggles, and a separate access-controlled store for sensitive payloads.

Q8. MEDIUM CLOUDFLARE

How do you attribute cost per feature, customer, or tenant in a multi-tenant LLM product?

Answer

Tag every span with tenant_id, feature, user_id, and model. Compute cost as input_tokens times input_price plus output_tokens times output_price per model, including cached-token discounts and tool-call costs. Aggregate along the tag dimensions and expose per-tenant dashboards with budget alerts. Without consistent tagging, cost attribution is impossible after the fact.

Q9. MEDIUM NOTION

What observability signals are unique to RAG pipelines?

Answer

Retrieval latency, number of chunks retrieved, similarity and retrieval scores, reranker scores, injected context length, retrieval hit or miss, and downstream groundedness. Crucially, log the actual retrieved chunk IDs so a bad answer can be traced back to bad retrieval versus bad generation, which are the two most common RAG failure modes.

Q10. MEDIUM ATLASSIAN

Design an observability dashboard for an AI assistant embedded in a SaaS product like Jira or Confluence.

Answer

Top-line: request volume, p50/p95/p99 latency, TTFT, error rate, and daily cost. Quality: thumbs up or down, deflection and resolution rate, sampled groundedness scores. Usage: tokens by model, top tools invoked, retrieval hit rate. Reliability: rate-limit hits, fallback rate, timeout rate. Every panel should be sliceable by workspace or tenant and by feature so you catch tenant-specific regressions.

Q11. HARD GOOGLE

OTel GenAI conventions cannot express faithfulness or groundedness. How do you close that gap architecturally?

Answer

OpenTelemetry is the data plane. It records that a call returned 1,200 tokens in 850ms but not whether those tokens contradict the source documents. Add a dedicated evaluation layer that consumes spans, runs faithfulness, toxicity, and policy scorers (model-based or LLM-judge), and writes scores back as span attributes or linked evaluation events. Keep these evals asynchronous and sampled so they do not inflate serving latency.

Q12. HARD META

At high QPS, capturing full prompt and response on every span is too expensive. Design a sampling strategy that preserves debuggability.

Answer

Combine head sampling for cost control, keeping full content on roughly 1 to 5 percent of traces, with tail sampling that always retains errors, high-latency, low-quality-score, and high-cost traces. Keep aggregated metrics at 100 percent since they are cheap. Retain lightweight span skeletons without content more broadly, and drive content capture with rules keyed on tenant, error, and score thresholds.

Q13. HARD DATABRICKS

How do you observe a multi-agent system where agents call other agents?

Answer

Model each agent invocation as a span; sub-agent calls nest as child spans under the caller via propagated context. Add agent.name, agent.role, and delegation edges, and use span links with from and to attributes for non-parent-child data flow where one agent's output feeds another. Track per-agent token, cost, and latency to find the slow or expensive agent and to detect loops, which show up as repeated identical spans.

Q14. HARD ARIZE

What is the execution graph view and why is it better than a flat span waterfall for agents?

Answer

A flat waterfall shows timing but hides data flow. An execution graph renders spans as nodes with edges for both parent-child relationships and span links, where output feeds input. It exposes branching, retries, tool-to-LLM data flow, and loops at a glance, which is essential for reasoning about the non-linear behaviour of agents that a linear timeline obscures.

Q15. EASY AMAZON

What is the difference between logs and traces?

Answer

Logs are discrete, timestamped events, often semi-structured. Traces are structured, causally-linked spans representing a request's full path, all sharing a trace_id. Traces give the request timeline and causality; logs give point-in-time detail. Structured logs can be attached to spans to combine both views.

Q16. MEDIUM ANTHROPIC

How do you observe token usage broken down by type: input, output, cached, and reasoning?

Answer

Modern models bill cached input tokens more cheaply and count reasoning tokens as part of output. Capture input_tokens, output_tokens, cache read and write tokens, and reasoning_tokens as separate span attributes; cost becomes a weighted sum. Collapsing these into one number makes cost dashboards materially wrong for reasoning-heavy or cache-heavy workloads.

Q17. MEDIUM LANGCHAIN

What are OpenLLMetry and OpenInference, and how do they relate to the OTel GenAI conventions?

Answer

Both are instrumentation SDKs that auto-emit spans for popular frameworks such as LangChain, LlamaIndex, and the OpenAI SDK. OpenLLMetry from Traceloop extends and helps lead the OTel GenAI conventions; OpenInference is Arize and Phoenix's schema. Choose one your backend accepts, because some platforms ingest OTel GenAI spans but not OpenInference, so schema compatibility drives the choice.

Q18. MEDIUM STRIPE

How do you observe streaming responses where tokens arrive incrementally?

Answer

Record time-to-first-token separately from total duration, plus inter-token latency. The span stays open until the stream completes, capturing finish_reason on close. Watch for stalls, which are long inter-token gaps, and truncation, which is a length finish_reason. For chat UIs, TTFT dominates perceived latency, so it deserves its own SLO.

Q19. HARD NETFLIX

You see p99 latency spikes only in production, never in tests. What observability data helps you root-cause it?

Answer

Break each trace into phases: admission or queueing, prefill (TTFT), decode, tool calls, and retrieval. Correlate spikes with model, tenant, prompt length, concurrent batch size (KV-cache pressure), tool timeouts, and provider-side latency. Tail-sampled full traces of the slow p99 requests, combined with GPU and queue-depth metrics, localize whether the culprit is your gateway, the model, or a downstream dependency.

Q20. HARD DATADOG

How do you build vendor-neutral observability so you are locked into neither one LLM provider nor one backend?

Answer

Instrument once with the OTel GenAI semantic conventions (gen_ai.*) and export via OTLP through an OpenTelemetry Collector. The Collector fans out via dual export to multiple backends and centrally applies redaction, sampling, and routing. Provider-agnostic attributes mean switching from OpenAI to Anthropic, or from Datadog to Phoenix, requires no re-instrumentation of your application.

Section 2: GenAI and Agent Traceability

20 questions on end-to-end distributed tracing for agents: trace and span structure, W3C Trace Context, MCP tool-call tracing, context propagation across async and protocol boundaries, span links, trajectory reconstruction, sampling at scale, and replay-from-trace regression testing.

Q1. EASY AMAZON

What is distributed tracing and why do agents need it?

Answer

Distributed tracing follows a single request across many services and steps via a shared trace_id and causally-linked spans. Agents chain numerous calls (LLM, tools, retrieval, sub-agents), so tracing reconstructs the full causal path, letting you see exactly what happened and in what order rather than staring at disconnected logs.

Q2. EASY GOOGLE

What are trace_id, span_id, and parent_span_id?

Answer

trace_id identifies the whole request; span_id identifies one operation; parent_span_id links a span to its caller, forming a tree. Together they allow a backend to reconstruct the complete execution tree from spans that were emitted independently by different components.

Q3. EASY MICROSOFT

What is W3C Trace Context and why does it matter for MCP?

Answer

W3C Trace Context is the standard for propagating trace identity across service boundaries via traceparent and tracestate HTTP headers. When both the MCP client and server propagate it, the server span nests under the client span, preserving trace continuity across the protocol boundary, so an agent's tool call and the MCP server's execution appear in one unified trace.

Q4. MEDIUM ANTHROPIC

How do you trace an MCP tool call end-to-end?

Answer

The client emits a tools/call span with gen_ai.tool.name, mcp.method.name set to tools/call, mcp.session.id, mcp.protocol.version, and network.transport (pipe for stdio, tcp for HTTP), plus jsonrpc.request.id. It propagates traceparent to the server, which creates a child span for the actual work. The result is one trace spanning agent to MCP client to MCP server to the underlying API.

Q5. MEDIUM UBER

How does context propagation work across async boundaries such as queues and background workers in an agent system?

Answer

The active span context must be captured at enqueue time and re-attached at dequeue time: inject trace context into the message, extract it on the worker. Otherwise the worker starts a new, disconnected trace. When strict parent-child does not apply, for example a batch job drawing from many source traces, use span links instead.

Q6. MEDIUM LANGCHAIN

What must a single agent step span contain to be replayable?

Answer

Inputs (messages and state), model and parameters, the reasoning or decision, the chosen tool and its arguments, the tool result, output messages, tokens, latency, and version stamps (prompt version, model version, code commit). With these you can deterministically replay or diff the step later during debugging or regression testing.

Q7. MEDIUM ATLASSIAN

A user reports a wrong answer from your AI assistant. Walk through using traces to debug it.

Answer

Find the session, then the specific trace for that turn. Inspect the span tree: was retrieval correct, meaning the right chunks? Did the model receive the right context? Which tool was called, with what arguments, and what did it return? Compare against a known-good trace of a similar request. This localizes the failure to retrieval, a tool, the prompt, or generation.

Q8. MEDIUM STRIPE

Why must you trace tool arguments and results, not just tool names?

Answer

The tool name tells you what was called; the arguments and results tell you whether it was called correctly and what it produced. Most agent failures are wrong arguments (hallucinated parameters) or misinterpreted results, both of which are invisible if you only record the tool name.

Q9. MEDIUM DATADOG

How do span links differ from parent-child relationships, and when do you use them?

Answer

Parent-child expresses direct causal nesting, meaning A called B. Span links express non-hierarchical relationships, such as a tool's output feeding a sibling LLM call, or a batch job derived from many source traces. Links carry from and to direction so the backend can draw data-flow edges in an execution graph rather than just a call tree.

Q10. MEDIUM NOTION

How do you correlate a trace with the exact prompt template and model version used?

Answer

Stamp each relevant span with prompt_template_id and version, model plus model_version, and the code or deploy version (git SHA). This lets you answer whether quality dropped because of the July prompt change or the model upgrade by filtering traces on version, turning a vague regression into a precise diff.

Q11. HARD META

Design trace sampling that stays useful for debugging while controlling storage cost at scale.

Answer

Combine head sampling, which is probabilistic, cheap, and decided at trace start, with tail sampling, which is decided after completion using the outcome: keep all errors, high-latency, low-eval-score, high-cost, or flagged-tenant traces. Keep 100 percent of metrics. Ensure the sampling decision propagates consistently so you never store partial traces, and run the tail sampler in the Collector.

Q12. HARD GOOGLE

How do you maintain trace continuity when a request crosses into a third-party LLM provider you do not control?

Answer

You cannot see inside the provider, so wrap the API call in a client span capturing request and response metadata: model, tokens, latency, and the provider's request-id. Store that request-id to correlate with the provider's side if you ever get support access. The provider is an opaque leaf span; everything before and after it stays in your trace.

Q13. HARD DATABRICKS

How do you trace and debug non-determinism, meaning same input but different output or path, in an agent?

Answer

Log the seed and temperature, model version, and full inputs per span. Because outputs vary, aggregate many traces of the same input to characterize the distribution of paths, and cluster trajectories to find divergent branches. Pin temperature to 0 and the model version to reproduce; if it is still non-deterministic, suspect provider-side batching or tool nondeterminism.

Q14. HARD ANTHROPIC

With MCP moving to stateless Streamable HTTP, what changes for traceability versus the old stateful SSE transport?

Answer

Old SSE was stateful with sticky sessions (Mcp-Session-Id), pinning a session to one instance, which was easy to correlate but hard to scale. Stateless Streamable HTTP makes each request self-contained, with context in headers and _meta, so any instance can serve any request. Traceability must therefore rely on propagated W3C trace context and per-request resource and session ids rather than server-held session state.

Q15. HARD NETFLIX

How do you trace a long-running or async agent task that outlives a single HTTP request?

Answer

Use a durable correlation id (task_id) plus the trace context stored with the task. Emit spans as work progresses (poll and step spans) linked back to the originating trace via span links. MCP's Tasks primitive, where the client polls for completion instead of holding a connection open, is the standardized pattern: each poll or step is traced under the task id rather than one long-held span.

Q16. EASY AMAZON

What is the difference between logging a request ID and true distributed tracing?

Answer

A request ID is a single correlation token you thread manually through logs. Distributed tracing adds structured spans, timing, causal parent-child links, and a standard propagation format, giving you a full execution tree and timeline, not just a value you can grep for across log lines.

Q17. MEDIUM SALESFORCE

How do you propagate user or tenant identity through a trace without leaking PII into every span?

Answer

Attach stable, non-PII identifiers such as tenant_id and a hashed user_id as span attributes or baggage for correlation, and keep raw PII out of span content or redact it. Baggage propagates across services but should carry only low-sensitivity keys, since it is often visible to every downstream service in the trace.

Q18. MEDIUM LANGCHAIN

What is a trajectory in agent evaluation, and how does tracing produce it?

Answer

A trajectory is the ordered sequence of steps an agent took: thought, tool call, observation, repeated, ending in a final answer. Tracing captures each step as a span; reading the span tree in order yields the trajectory, which is exactly what trajectory-level evaluation and debugging operate on.

Q19. HARD UBER

How would you build a replay-from-trace capability for regression testing agents?

Answer

Capture inputs, tool results, model parameters, and versions per span. To replay, re-run the agent while feeding the recorded tool results, mocking external calls, so you isolate model and prompt behaviour from flaky externals. Diff the new trajectory against the recorded one and flag divergences. This turns real production traces into an automated regression suite.

Q20. HARD DATADOG

How do you keep trace overhead, in latency and cost, acceptable in a high-throughput production agent?

Answer

Use async, batched, non-blocking span export; sample content-heavy attributes; avoid synchronous flushes on the hot path; cap attribute sizes; and offload redaction and sampling to the Collector. Measure the instrumentation's own overhead and target under 1 to 2 percent added latency. Prefer cheap always-on metrics for baseline signals and reserve full traces for sampled deep dives.

Section 3: Monitoring AI Systems

20 questions on keeping production AI healthy: SLOs and error budgets, drift detection, hallucination and guardrail monitoring, silent provider upgrades, cost-anomaly control, alerting without fatigue, seasonality-aware anomaly detection, and closing the incident-to-fix loop.

Q1. EASY AMAZON

What SLOs would you define for an LLM-powered API?

Answer

Availability, for example 99.9 percent success; latency, such as p95 TTFT below a target and p95 total below a target; error rate; and optionally a quality SLO such as sampled groundedness above a threshold. Attach an error budget to each and alert on burn rate rather than on every individual breach.

Q2. EASY MICROSOFT

What is model or data drift and why does it matter in production?

Answer

Drift is when the live data distribution diverges from the training or eval distribution: covariate drift (inputs change), concept drift (the input-to-output relationship changes), and prediction drift (output distribution shifts). It silently degrades quality with no code change, so it must be monitored continuously rather than assumed stable.

Q3. EASY DATADOG

What is the difference between monitoring infrastructure metrics and monitoring model quality?

Answer

Infra metrics such as GPU utilisation, latency, error rate, and queue depth tell you the system is up and fast. Quality monitoring such as groundedness, hallucination rate, and user feedback tells you the answers are good. A system can be perfectly healthy on infrastructure while producing nonsense, so you need both layers.

Q4. MEDIUM META

How do you monitor hallucination rate in production without ground-truth labels?

Answer

Sample live traffic and run reference-free scorers: groundedness and faithfulness checks against retrieved context (NLI or LLM-judge), self-consistency across samples, and citation verification. Track the score distribution over time and alert on regressions. Complement with implicit user feedback such as thumbs-down, edits, and escalations as a weak label.

Q5. MEDIUM NETFLIX

How do you detect a silent model-provider upgrade degrading your app?

Answer

Record model_version per request and alert on changes. Run a small canary eval set of golden prompts continuously and watch score deltas, plus output-distribution metrics like length, refusal rate, and format validity for step changes. A sudden shift in judge scores or refusal rate with no deploy on your side implicates the provider.

Q6. MEDIUM STRIPE

What guardrail metrics should you monitor for a customer-facing agent?

Answer

Refusal and over-refusal rate, jailbreak and prompt-injection detection hits, PII-leak detections, toxicity and safety-filter triggers, tool-permission denials, and out-of-scope rate. Monitor both false negatives, meaning harmful content slipping through, and false positives, meaning blocking legitimate requests, since over-blocking silently hurts usefulness.

Q7. MEDIUM ATLASSIAN

How do you monitor AI-feature quality at the tenant or workspace level in a SaaS product?

Answer

Slice every quality, usage, and cost metric by tenant: feedback rate, resolution and deflection, sampled groundedness, cost per user, error rate. This surfaces tenant-specific regressions, for instance a customer whose documents cause bad retrieval, that are invisible in global averages. Add per-tenant anomaly detection.

Q8. MEDIUM LINKEDIN

What is an embedding-drift monitor and how does it work?

Answer

Embed incoming queries or retrieved documents and compare their distribution to a baseline using metrics like population stability index, MMD, or cosine distance to cluster centroids. Rising divergence signals topic or domain drift that can quietly degrade retrieval and generation before any user complaint arrives.

Q9. MEDIUM DATABRICKS

How do you monitor a RAG system's retrieval quality in production?

Answer

Track retrieval scores, hit rate (did any chunk clear a relevance threshold?), context precision and recall on sampled labelled queries, stale-document rate, and downstream answer groundedness. Alert when retrieval scores drop, often from index drift or new document types, even when latency and error metrics look perfectly fine.

Q10. MEDIUM UBER

Describe an alerting strategy that avoids alert fatigue for an AI system.

Answer

Alert on symptoms users actually feel, such as SLO burn rate, error spikes, and sustained quality regressions, not on every raw metric. Use multi-window burn-rate alerts, dynamic or anomaly thresholds rather than static ones, deduplicate, set severities, and attach runbooks. Quality alerts should fire on statistically significant sustained drops, never on a single bad sample.

Q11. HARD GOOGLE

Design an end-to-end monitoring system for a production LLM app covering infra, cost, and quality.

Answer

Four layers. Infra: latency (TTFT and total), throughput, error rate, GPU and queue metrics, rate-limit hits. Cost: tokens and cost by model, tenant, and feature with budgets. Quality: sampled online evals (groundedness, toxicity, task success), user feedback, guardrail hits. Drift: input-embedding drift, output distribution, model-version tracking. Pipe everything via OTel; add dashboards, SLOs, burn-rate alerts, and anomaly detection; feed confirmed failures into the eval regression set.

Q12. HARD META

How do you monitor quality when there is no immediate ground truth and human labelling is slow and expensive?

Answer

Rely on proxy and implicit signals continuously: a calibrated LLM-judge on a traffic sample, behavioural signals (edits, retries, thumbs, escalation to human, task completion), and reference-free metrics. Reserve human labelling for calibration and hard cases, and track judge-human agreement so you know how much to trust the proxy.

Q13. HARD NETFLIX

What statistical care is needed before alerting on a quality drop?

Answer

A few bad samples are not a regression. Require statistically significant deltas: adequate sample size, confidence intervals, multi-window confirmation, and control for confounders like traffic-mix, tenant, or prompt-length shifts. Beware the multiple-comparisons problem when monitoring many slices at once. Alert only on sustained, significant burn.

Q14. HARD DATABRICKS

How do you monitor and control cost anomalies such as runaway agent loops in real time?

Answer

Enforce per-request budgets (max tokens, max tool calls, max steps, max wall-clock) and interrupt on breach. Monitor tokens-per-request and steps-per-request distributions, alerting on tail growth, and detect loops via repeated identical spans. Aggregate spend with real-time per-tenant budget alerts, and add circuit breakers that degrade to a cheaper model or refuse.

Q15. HARD ANTHROPIC

How do you monitor for prompt-injection and jailbreak attempts across production traffic?

Answer

Run input and output classifiers for injection patterns and policy violations, logging each detection with its trace. Monitor detection-rate trends and cluster novel patterns of blocked prompts. Critically, watch for indirect injection via retrieved or tool content, not just direct user input. Alert on spikes and feed confirmed attacks into your red-team and eval sets.

Q16. EASY AMAZON

What is a canary deployment and how does it help monitor AI changes?

Answer

Route a small percentage of traffic to the new model or prompt, compare its latency, error, quality, and cost against the control, and roll forward only if it stays healthy. This limits blast radius and gives a live production signal before a full rollout, which is far safer than an all-at-once switch.

Q17. MEDIUM SALESFORCE

How do you monitor whether users actually find AI outputs useful, not just that the system is up?

Answer

Track product signals: acceptance and edit rate for suggestions, thumbs up or down, copy and apply actions, task completion, follow-up and retry rate, deflection (issues resolved without a human), and feature retention. These behavioural signals proxy real usefulness far better than infrastructure health, which can be green while users quietly abandon the feature.

Q18. MEDIUM CONFLUENT

How do you monitor tool-call reliability in an agent?

Answer

Track per-tool call volume, success and error rate, latency, timeout rate, retry rate, and argument-validation failures. Alert when a specific tool's error rate spikes, usually an upstream API change, because one flaky tool can cascade into broad agent failures that look like model problems but are not.

Q19. HARD UBER

How would you build anomaly detection for AI-system metrics that have strong seasonality?

Answer

Use seasonal baselines (per hour-of-day and day-of-week) via decomposition such as STL or Prophet-style models, and alert on residuals beyond dynamic bounds rather than static thresholds. Normalize traffic-driven metrics per-request, and add change-point detection to catch step shifts from deploys or provider changes that seasonal models would otherwise absorb.

Q20. HARD DATADOG

How do you close the loop from a monitoring alert back to a fix and prevent recurrence?

Answer

On alert, use the linked traces to root-cause (retrieval, tool, prompt, or model). Fix it, then add the failing case to the eval regression suite so future changes are gated on it. If the cause is drift, trigger re-indexing or re-tuning. Track MTTR and recurrence; a healthy loop turns every incident into a permanent regression test.

Section 4: AI Evals

20 questions on evaluating AI quality: offline versus online evals, golden datasets, LLM-as-a-judge and its biases, RAGAS metrics, agent trajectory and tool-use evaluation, judge calibration, statistical rigor, safety and red-team evals, and continuous production evaluation.

Q1. EASY OPENAI

What is the difference between offline and online evaluation?

Answer

Offline evaluation runs against a fixed dataset before deploy, acting as a regression gate and a way to compare models and prompts. Online evaluation scores live production traffic (sampled) to catch drift and real-world failures that offline sets miss. You need both: offline to ship safely, online to stay safe as reality shifts.

Q2. EASY MICROSOFT

What is a golden dataset and how do you build one?

Answer

A curated, version-controlled set of representative inputs with expected outputs or criteria, used for regression testing. Build it by promoting interesting and failing cases from real production traces, covering key scenarios and edge cases, keeping it decontaminated from training data, and growing it every time a new failure is found.

Q3. EASY GOOGLE

What is LLM-as-a-judge and when is it appropriate?

Answer

Using an LLM to score outputs against a rubric (direct scoring) or to pick the better of two (pairwise). It is appropriate for qualitative dimensions such as helpfulness, faithfulness, and tone that are hard to check deterministically. It is the wrong tool when a deterministic check like schema validity, exact match, or numeric tolerance already suffices.

Q4. MEDIUM RAGAS

Explain RAGAS's four core RAG metrics.

Answer

Faithfulness (is the answer grounded in the retrieved context?), Answer Relevancy (does it actually address the question?), Context Precision (are the retrieved chunks relevant and well-ranked?), and Context Recall (did retrieval fetch all the information needed?). Together they separate retriever failures from generator failures. RAGAS v0.4+ extends to agents with Topic Adherence and Agent Goal Accuracy metrics.

Q5. MEDIUM META

How do you detect and mitigate position bias in LLM-as-a-judge?

Answer

Position bias is when the judge favours the first or second response regardless of quality. Detect it by presenting the same pair twice with the order swapped; if the verdict flips, bias is confirmed. Mitigate by averaging both orderings, randomizing position, or switching from pairwise to rubric-based direct scoring.

Q6. MEDIUM ANTHROPIC

Why must you lock the judge model version, and what is temporal drift in evals?

Answer

If the judge model silently updates, its scoring shifts and your eval numbers move with no change to your system, which is temporal drift. Pin the judge model version, treat the judge itself as a system under test, and re-calibrate against human labels whenever you are forced to upgrade it.

Q7. MEDIUM DEEPEVAL

How does evaluation-as-unit-testing work in CI/CD?

Answer

Write eval cases as tests (pytest-style) with assertions on metric thresholds, for example faithfulness at least 0.8. Run them in CI on every prompt, model, or retrieval change and block the merge on regressions. DeepEval's DAG metric structures grading as a decision tree for deterministic, customizable scoring, making LLM behaviour testable like ordinary code.

Q8. MEDIUM LANGCHAIN

How do you evaluate an agent's trajectory, not just its final answer?

Answer

Score the process: tool-selection accuracy (right tool?), tool-argument correctness, step efficiency (any wasted steps?), goal progress, and adherence to allowed actions. Agent-as-a-Judge inspects the full reasoning trace. Two agents can reach the same answer via very different good or bad paths, so trajectory eval catches process failures an outcome-only metric hides.

Q9. MEDIUM SCALE AI

How do you calibrate an LLM judge against humans, and what agreement is acceptable?

Answer

Collect at least around 100 human-labelled examples per rubric and measure judge-human agreement (accuracy, Cohen's kappa, correlation). Around 75 percent or higher agreement is usable for trend monitoring; below about 65 percent the judge adds more noise than signal. Recalibrate whenever the rubric or the judge model changes.

Q10. MEDIUM ATLASSIAN

You changed your chunking strategy. How do you prove it improved the system?

Answer

Run the golden RAG eval set through the old and new configs and compare context precision and recall, faithfulness, answer relevancy, and end-task success with confidence intervals. Ensure the eval set is representative and decontaminated. Then A/B a traffic slice online to confirm the offline win actually holds in production.

Q11. HARD GOOGLE

Design an eval framework for a production RAG agent from scratch.

Answer

Define failure modes (hallucination, bad retrieval, wrong tool, unfaithfulness). Build a small representative golden set from real traces. Choose metrics per layer: retrieval (precision and recall), generation (faithfulness and relevancy), agent (tool accuracy, trajectory, goal accuracy). Mix evaluators: deterministic checks, a calibrated LLM-judge, and human review for high-risk cases. Gate releases offline and run streaming online evals on sampled traffic. Finally, turn every production failure into a regression case.

Q12. HARD META

What are the biggest ways LLM-judge evals silently give wrong numbers?

Answer

Data contamination (eval set in the training corpus inflates scores), position, verbosity, and self-preference bias, an uncalibrated judge, temporal drift from judge upgrades, a weak or mismatched judge model, single-metric tunnel vision, and tiny or unrepresentative eval sets. Each quietly corrupts results; guard with decontamination, bias tests, calibration, version pinning, and composite metrics.

Q13. HARD NETFLIX

How do you evaluate open-ended generation where there is no single correct answer?

Answer

Use reference-free rubric scoring (an LLM-judge on defined criteria), pairwise comparison against a baseline with order-swapping, and multi-dimensional metrics (coherence, relevance, safety, style). Anchor everything with periodic human evaluation. Report distributions and win-rates versus the baseline rather than a single misleading accuracy number.

Q14. HARD DATABRICKS

How do you make evals statistically rigorous when comparing two models or prompts?

Answer

Use a sample large enough for statistical power, report confidence intervals, and run significance tests: paired tests since both see the same inputs, and bootstrap for judge scores. Control for multiple comparisons across metrics and slices. For pairwise judge win-rates, account for ties and position bias, and never ship on a point-estimate delta that sits inside the noise.

Q15. HARD OPENAI

How do you evaluate tool-use and function-calling correctness?

Answer

Metrics: tool-selection accuracy (right tool for the intent), argument correctness (schema-valid and semantically right values), call necessity (did not call when unneeded, did call when needed), and end-task success. Use deterministic checks on the structured calls plus judge or human review for judgment calls, built from a suite of intents with expected tool and argument pairs.

Q16. EASY AMAZON

What is regression testing for prompts and why is it essential?

Answer

Re-running your golden eval set after any prompt, model, or retrieval change to ensure you did not break previously-working cases. It is essential because LLM changes are non-local: fixing one case often silently breaks others. Every confirmed production failure should become a permanent regression case in the suite.

Q17. MEDIUM ARIZE

What does OTel-native evals attached to traces give you?

Answer

Because evals attach to trace spans, you can score real production runs in place, filter and aggregate scores by model, tenant, and prompt-version, and jump from a low score straight to the offending trace for debugging, regardless of which runtime framework produced the span. It unifies observability and evaluation instead of running them as separate silos.

Q18. MEDIUM STRIPE

How do you evaluate safety and guardrails, and why is red-teaming part of evals?

Answer

Maintain adversarial test sets (jailbreaks, injections, PII-extraction attempts, harmful-request prompts) and measure attack-success rate alongside over-refusal rate. Red-teaming generates new attacks that become eval cases. Safety evals must cover both missing real harms (false negatives) and blocking benign requests (false positives).

Q19. HARD ANTHROPIC

What is Agent-as-a-Judge and when does it beat a plain LLM judge?

Answer

An agentic evaluator that uses multi-step reasoning, state, and tools to inspect another agent's full trajectory, analysing intermediate artifacts, building reasoning graphs, and validating hierarchical requirements. It beats an output-only LLM judge when the process matters as much as the result, for example code-generation agents on the DevAI benchmark, aligning more closely with human experts on complex, process-oriented tasks.

Q20. HARD UBER

How do you continuously evaluate production traffic without hurting latency or blowing up cost?

Answer

Run online evals asynchronously on a sampled subset rather than inline. Use cheaper, faster judge models for high-throughput trend monitoring and stronger judges for periodic deep grading, batch eval jobs off the hot path, and store scores on the traces. Prioritize sampling toward risky, low-confidence, and high-cost traffic where evaluation pays off most.

Section 5: Deployment of AI Apps (GenAI, MCP, Agents)

45 questions across four sub-themes: LLM serving and inference infrastructure, MCP server deployment (transports, OAuth 2.1, stateless scaling, Tasks), agent deployment (state, budgets, safety, multi-agent), and gateways, reliability, and CI/CD for shipping AI changes safely at scale.

LLM Serving and Inference Infrastructure

Q1. EASY AMAZON

What are the main options for deploying an LLM: hosted API versus self-hosted?

Answer

Hosted API (OpenAI, Anthropic, Bedrock) is fastest, with no GPU operations and pay-per-token pricing, but offers less control and raises data-residency and vendor-lock concerns. Self-hosted (vLLM or TGI on your own GPUs) gives full control, keeps data in-house, and is cheaper at high scale, but you own scaling, reliability, and GPU cost. Many teams start hosted and self-host once volume justifies it.

Q2. EASY NVIDIA

What is vLLM and why is it popular for serving?

Answer

An open-source inference server built around PagedAttention (KV-cache paging) and continuous batching to maximize GPU throughput. It packs many concurrent requests efficiently, supports streaming, quantization, and tensor parallelism, and delivers high tokens per second with strong utilisation compared to naive serving loops.

Q3. EASY GOOGLE

What is continuous or in-flight batching and why does it matter for deployment?

Answer

Instead of waiting to assemble a static batch, the server adds and removes requests from the running batch as sequences finish and new ones arrive. This keeps the GPU continuously busy, improving throughput and cutting queueing latency under mixed-length workloads, which is a key reason modern servers beat request-at-a-time inference.

Q4. MEDIUM META

How do you autoscale GPU-backed LLM inference, and why is it harder than autoscaling stateless web services?

Answer

GPUs are expensive, slow to start (a cold start is node provisioning plus loading the model into VRAM, often minutes), and memory-bound by the KV cache. Scale on queue depth, tokens-in-flight, or GPU utilisation rather than CPU. Keep warm pools to avoid cold starts, add admission control and request queueing, scale in coarse steps, and consider disaggregating prefill and decode pools.

Q5. MEDIUM DATABRICKS

How do you deploy many fine-tuned variants cost-effectively?

Answer

Serve LoRA adapters on top of a single base model (multi-LoRA serving): base weights are shared in VRAM while adapters are swapped or batched per request. This serves many task-specific variants on one GPU instead of one full model per variant. Merge an adapter into the base only when you need maximum single-model speed for that variant.

Q6. MEDIUM CLOUDFLARE

How do quantized models change your deployment footprint and trade-offs?

Answer

INT8 and INT4 quantization (GPTQ, AWQ) shrinks VRAM and speeds up memory-bandwidth-bound decoding, letting large models fit on smaller, cheaper GPUs. The trade-off is a small accuracy loss, worse at 4-bit for small models, plus kernel and hardware support requirements. It is ideal for cost-sensitive or edge deployment, but validate quality with your eval suite before shipping.

Q7. MEDIUM NETFLIX

Compare blue-green, canary, and shadow deployment for shipping a new model or prompt.

Answer

Blue-green keeps a full standby environment for an instant switch and rollback. Canary sends a small percentage of live traffic and ramps up gradually behind metric gates. Shadow mirrors traffic to the new version without serving its output to users, so you compare quality and latency safely. Use shadow for correctness, canary for gradual live rollout, and blue-green for fast rollback.

Q8. MEDIUM UBER

How do you handle streaming responses in production infra: load balancers and timeouts?

Answer

Use SSE or WebSocket with long-lived connections; raise LB and proxy idle timeouts, disable response buffering, set TCP_NODELAY, and flush per token. Track TTFT separately from total duration. Handle client disconnects gracefully by stopping generation to save cost, and support reconnection so a dropped stream does not force a full re-run.

Q9. HARD META

Design the serving stack to deliver an LLM to millions of users under tight latency SLOs.

Answer

Model layer: quantize, enable KV cache, run vLLM or TGI with PagedAttention and continuous batching, and disaggregate prefill and decode pools to avoid head-of-line blocking. Gateway: load balancer, rate limiting, semantic cache, prompt router, fallbacks. Scaling: Kubernetes with GPU autoscaling and warm pools, multi-region for latency and failover. Reliability: circuit breakers, timeouts, retries with backoff. Observability: TTFT, TPS, cost, and error dashboards with SLO burn alerts.

Q10. HARD NVIDIA

How do you optimize for both throughput and latency when they pull in opposite directions?

Answer

Throughput wants large batches; latency wants small ones. Balance them with continuous batching (fills the GPU without forcing large static batches), priority queues (interactive versus batch), speculative decoding (cuts per-step latency), KV-cache quantization (fits more sequences), and prefill and decode disaggregation. Set per-tier SLOs and route accordingly, and run offline or async work on a separate path from interactive traffic.

MCP (Model Context Protocol) Deployment

Q11. EASY ANTHROPIC

What are MCP's two transports, and when do you use each?

Answer

stdio is local and process-bound: the host spawns the server as a subprocess communicating over stdin and stdout, which is the default for local development and Claude Desktop. Streamable HTTP is a network transport over a single HTTP endpoint, required for remote hosting, multiple concurrent clients, and container orchestration.

Q12. EASY MICROSOFT

Why was HTTP+SSE deprecated in favour of Streamable HTTP?

Answer

HTTP+SSE needed two endpoints (initialization plus messages) and sticky sessions, which broke load balancing and made horizontal scaling impractical, with no standard security model. Streamable HTTP uses a single endpoint, is stateless-capable, and works behind standard load balancers and proxies, so it replaced SSE, which was deprecated in the 2025-06-18 spec and is being sunset across providers in 2026.

Q13. MEDIUM ANTHROPIC

Why does stdio fall apart when you try to deploy an MCP server to the cloud?

Answer

stdio assumes one client spawning one local subprocess over pipes. It cannot handle concurrent connections, network access, or container orchestration; there is no networking, no multi-client fan-out, and no path to horizontal scaling. Remote deployment therefore requires Streamable HTTP, proper authentication, and scalable infrastructure.

Q14. MEDIUM OKTA

Walk through the OAuth requirements for a remote MCP server.

Answer

HTTP transports require OAuth 2.1 with mandatory PKCE. The MCP server acts as an OAuth Resource Server: it exposes /.well-known/oauth-protected-resource (RFC 9728) so clients can discover the authorization server and supported scopes. Clients must use Resource Indicators (RFC 8707) so tokens are audience-bound to that specific server URL; a stolen token cannot be replayed against a different MCP server because the audience check fails.

Q15. MEDIUM RENDER

Why is stateless operation important for scaling MCP servers, and how do you achieve it?

Answer

Stateful servers need sticky sessions, pinning each client to one instance and blocking horizontal scaling. Stateless Streamable HTTP makes each request self-contained, with context carried in headers and _meta, so any instance behind a standard load balancer can serve any request. You then scale MCP servers exactly like REST APIs (Kubernetes pods, serverless functions, edge workers) with autoscaling and no session affinity.

Q16. MEDIUM ATLASSIAN

How would you deploy an internal MCP server exposing company tools like Jira or Confluence to AI clients securely?

Answer

Run Streamable HTTP behind your gateway, with OAuth 2.1 plus PKCE and audience-bound tokens (RFC 8707) so tokens only work for this server. Scope tools by least privilege per user, validate and redact tool inputs and outputs, rate-limit, and log every tools/call with trace context. Run stateless for scaling, and treat all retrieved and tool content as untrusted to defend against indirect prompt injection.

Q17. MEDIUM STRIPE

What security risks are unique to hosting MCP servers, and how do you mitigate them?

Answer

Risks include over-broad tool permissions, token replay across servers, prompt and tool-response injection, unauthenticated access, and confused-deputy problems. Mitigate with OAuth 2.1 plus PKCE, audience-bound tokens (RFC 8707), least-privilege scopes per tool, input validation, output sanitization, human approval for destructive actions, and comprehensive audit logging.

Q18. MEDIUM CLOUDFLARE

How do you deploy MCP servers on serverless or edge platforms, and what are the constraints?

Answer

Stateless Streamable HTTP maps cleanly onto serverless functions and edge workers since each request is self-contained. Constraints: cold starts, execution-time limits (long-running tools need the Tasks poll pattern rather than a held connection), and per-request auth overhead. Keep tools fast and idempotent, and use the Tasks primitive for anything asynchronous.

Q19. MEDIUM ANTHROPIC

What is the MCP Tasks primitive and why does it matter for deployment?

Answer

Tasks, introduced experimentally around the 2025-11-25 spec, support long-running or asynchronous operations that exceed a normal HTTP request lifetime: instead of holding a connection open, clients poll for completion. This lets stateless HTTP infrastructure handle slow tools without long-lived connections, which matters for serverless and scale-out. Treat it as forward-looking until it stabilizes.

Q20. HARD MICROSOFT

Design a production deployment for a fleet of remote MCP servers used by many agents across an enterprise.

Answer

Streamable HTTP, stateless, behind an API gateway and load balancer, on Kubernetes with horizontal autoscaling. OAuth 2.1 plus PKCE, per-server audience-bound tokens, a centralized authorization server, and per-tool least-privilege scopes. Maintain a registry or catalog of approved servers and pin mcp.protocol.version. Observability: trace every tools/call with W3C context, mcp.session.id, and per-tool latency and error SLOs. Add guardrails against indirect injection, audit logs, and blue-green upgrades.

Q21. HARD DATABRICKS

How do you handle versioning and backward compatibility as the MCP spec evolves, for example SSE sunset and stateless migration?

Answer

Pin and advertise mcp.protocol.version and negotiate it during initialization. Run old and new transports in parallel during migration, keeping SSE clients served until sunset while defaulting new clients to Streamable HTTP. Gate on capability negotiation, use feature flags, and monitor the client version distribution so you can deprecate the old transport safely once traffic has moved off it.

Q22. HARD META

How do you trace and debug failures across the agent to MCP client to MCP server boundary in production?

Answer

Propagate W3C Trace Context so the server span nests under the client span in one trace. Capture mcp.method.name, mcp.session.id, mcp.protocol.version, gen_ai.tool.name, network.transport, and jsonrpc.request.id. On failure, follow the trace from the agent's tool decision to client tools/call to server execution to the downstream API to localize whether the cause is bad arguments, auth, tool logic, or the upstream service.

Agent Deployment

Q23. EASY AMAZON

What extra deployment concerns do agents add beyond a single LLM call?

Answer

Multi-step loops that must be bounded for cost, tool integrations (auth and reliability), memory and state persistence, non-determinism, longer latency, and safety since agents take real actions. You must add step and cost caps, tool sandboxing, external state stores, retries and timeouts, and human-approval gates for irreversible actions.

Q24. EASY LANGCHAIN

Where does agent memory and state live in a deployed system?

Answer

Short-term conversation state lives in a request or session store (Redis, a database) keyed by session or thread id; long-term memory lives in a vector store or database. The agent runtime should be stateless per instance and load and save state externally, so any instance can handle any turn and the service scales horizontally.

Q25. MEDIUM UBER

How do you make agent runs stateless and horizontally scalable?

Answer

Externalize all state (conversation, memory, task progress) to shared stores keyed by session or task id, and keep the runtime process stateless. Each turn: load state, run, persist state. This lets a load balancer send turns to any instance and enables autoscaling, restarts, and failover without losing context mid-conversation.

Q26. MEDIUM STRIPE

How do you enforce cost and safety limits on autonomous agents in production?

Answer

Set hard per-run caps (max steps, max tool calls, max tokens, max wall-clock) and interrupt on breach. Sandbox tool execution with least privilege, allowlist tools, and require human-approval gates for irreversible or destructive actions. Add circuit breakers that degrade to a cheaper model or refuse, and monitor for loops (repeated spans) and per-run cost distributions.

Q27. MEDIUM NETFLIX

How do you handle tool and API failures gracefully in a deployed agent?

Answer

Use timeouts plus retries with exponential backoff and jitter, circuit breakers on flaky tools, and fallbacks (an alternate tool, a cached result, or graceful degradation). Validate tool outputs before feeding them back to the model, and surface a clean error or finish rather than looping. Trace every tool call so you can root-cause upstream failures quickly.

Q28. MEDIUM ATLASSIAN

How would you deploy an agent that acts inside a SaaS product, creating tickets or editing docs, safely?

Answer

Scope permissions to the acting user with no privilege escalation, require confirmation for writes and destructive actions, and run tools with per-tenant least privilege. Validate and preview actions, and audit-log every action with its trace. Rate-limit, sandbox, and treat all document and tool content as untrusted input to prevent indirect prompt injection from turning your agent against the user.

Q29. MEDIUM DATABRICKS

How do you deploy multi-agent systems and manage their failure modes?

Answer

Use an orchestrator and worker pattern with structured message contracts, bounded delegation depth, and a global step and cost budget. Add per-agent timeouts, loop detection, a critic or verifier agent for checkpoints, and full tracing with sub-agent spans nested under callers. Guard against the classic failure modes: error cascades, ping-pong loops, conflicting instructions, and context fragmentation across agents.

Q30. MEDIUM OPENAI

How do you version and roll back agents (prompt, tools, model) as a unit?

Answer

Treat the agent configuration (prompt templates, tool schemas, model and parameters, orchestration graph) as versioned artifacts bundled under one release version. Deploy via canary or shadow, stamp every trace with the version, and roll the whole bundle back atomically. Gate releases on the eval regression suite so a prompt or tool change cannot silently regress behaviour.

Q31. HARD GOOGLE

Design a production deployment for a customer-facing agent with tools, memory, and strict latency and safety needs.

Answer

Stateless runtime with external session, memory, and task stores. Tool layer: sandboxed, least-privilege, timeouts, retries, circuit breakers, and human gates for writes. Guardrails: input and output filters for injection and PII, allowlisted tools. Budgets: step, tool, token, and time caps. Serving: streaming for TTFT, a model gateway with fallbacks, and autoscaling. Observability and evals: full trajectory tracing, online quality and safety evals, and SLO alerts. Rollout: canary plus shadow with atomic versioned rollback.

Q32. HARD META

How do you deploy an agent for low latency when it makes many sequential LLM and tool calls?

Answer

Parallelize independent tool calls and sub-tasks, execute likely tools early or speculatively, and cache retrievals and repeated LLM calls with a semantic cache. Use smaller, faster models for routing and simple steps while reserving the large model for hard steps, stream partial results, prefetch likely-needed data, and bound the plan depth. Trace phase timings so you attack the single biggest latency contributor first.

Q33. HARD ANTHROPIC

How do you defend a deployed agent against indirect prompt injection from tool or retrieved content?

Answer

Treat all tool, retrieved, and document content as untrusted data, never as instructions. Separate instruction context from data context, use spotlighting and delimiters, constrain tool permissions to least privilege, require human approval for high-impact actions, run output and action validators, and monitor for injection patterns in tool responses. Assume any external text your agent reads may be adversarial.

Q34. HARD UBER

How do you load-test and capacity-plan an agentic system before launch?

Answer

Model realistic trajectories (variable step counts, tool latencies, prompt lengths), not single calls. Load-test at production-like concurrency, measuring p95 and p99 end-to-end latency, tokens and steps per run, GPU and queue saturation, tool-dependency limits, and cost per run. Find the bottleneck (model, tool, or KV cache), set autoscaling policies and per-run budgets, and validate failover under partial tool outages.

Gateways, Reliability and CI/CD

Q35. EASY CLOUDFLARE

What is an LLM or model gateway and why deploy one?

Answer

A middleware layer between apps and model backends that provides a unified API, routing and load-balancing across providers and models, auth, rate limiting, caching, cost tracking, logging, and fallbacks. It centralizes cross-cutting concerns so applications do not hardcode provider logic, and it lets you switch or route models without changing application code.

Q36. EASY AMAZON

What is rate limiting and why is it critical for AI deployments?

Answer

Capping requests or tokens per client or tenant over a time window to protect capacity, control cost, and ensure fairness. It is critical because LLM calls are expensive and GPU capacity is finite; without limits, one tenant or a runaway loop can exhaust capacity or budget. Implement per-tenant token and request quotas with backpressure.

Q37. MEDIUM NETFLIX

How do fallback chains and circuit breakers improve LLM app reliability?

Answer

A fallback chain retries a backup model or provider (or a cached or degraded response) on error, timeout, or rate-limit, so users still get an answer. A circuit breaker, after repeated failures to a backend, temporarily stops sending it traffic to avoid cascading failures and give it time to recover. Together they turn a provider outage into graceful degradation instead of a full incident.

Q38. MEDIUM RAMP

How does semantic caching work and what are its risks?

Answer

It caches responses keyed by embedding similarity of prompts rather than exact match, so a new prompt close to a cached one returns the cached answer, cutting cost and latency. Risks: false cache hits (semantically close but materially different prompts get the wrong answer), staleness, and cross-tenant privacy leakage. Mitigate with tight similarity thresholds, tenant-scoped caches, and TTLs.

Q39. MEDIUM MICROSOFT

How do you manage prompts as deployable, versioned artifacts (PromptOps)?

Answer

Store prompts in version control with IDs and versions, review changes, and deploy them through CI/CD like code. Stamp traces with prompt_version, gate changes on eval regression tests, support A/B and canary of prompt versions, and enable instant rollback. Never edit production prompts ad hoc, because that is how silent quality regressions ship.

Q40. MEDIUM STRIPE

How do you secure secrets and API keys in AI deployments?

Answer

Store provider keys in a secrets manager (Vault, KMS), never in code or prompts, and inject them at runtime. Scope and rotate keys, use per-tenant or per-service keys for attribution and blast-radius control, and monitor for key leakage in logs and outputs. Redact secrets from all telemetry so they never land in a trace.

Q41. MEDIUM DATABRICKS

How do you deploy embeddings and vector search for RAG at scale?

Answer

Batch-embed and index documents into a scalable vector DB with metadata and access control, serve ANN queries with replicas for QPS, and handle incremental re-indexing as documents change. Version the embedding model and re-embed on model change, monitor retrieval latency and recall, and co-locate the index with the app region to keep retrieval latency low.

Q42. MEDIUM ATLASSIAN

What CI/CD gates should exist before an AI change reaches production?

Answer

Lint and tests, a prompt and agent eval regression suite with quality gates (faithfulness, task success), safety and red-team checks, latency and cost budget checks, schema and tool-contract validation, and a staged rollout (shadow to canary to full) with automated rollback on metric breach. No AI change should ship without passing the eval gates.

Q43. HARD GOOGLE

Design a multi-region, highly-available deployment for a global GenAI product.

Answer

Deploy inference, gateway, and vector store per region for latency and data residency, routing users to the nearest healthy region via GeoDNS or anycast. Add cross-region failover with health checks and circuit breakers, replicate indexes and config, and apply region-aware rate limits and budgets. Handle provider-region outages with fallback providers, keep observability and evals consistent across regions, and use per-region blue-green for safe upgrades.

Q44. HARD META

How do you roll out a risky model or prompt change to a billion-user product with confidence?

Answer

Use a layered rollout: an offline eval gate, then shadow (mirror traffic, compare quality, latency, and cost with no user impact), then canary (a tiny metric-gated percentage), then a progressive rollout by region or tenant, all with automated rollback on SLO, quality, or cost burn. Stamp versions on traces, run online evals on the canary, and keep blue-green available for instant revert.

Q45. HARD DATADOG

How do you unify deployment, observability, and evaluation into one production feedback loop?

Answer

Instrument with the OTel GenAI conventions so every request emits traces, metrics, and cost. Attach online evals to sampled traces (quality and safety scores on spans). Monitor SLOs, drift, and cost with burn-rate alerts. Promote interesting and failing production traces into versioned golden datasets, gate every deploy on the eval suite, canary or shadow new versions, and auto-rollback on regression. The loop (deploy, observe, eval, curate datasets, gate the next deploy) means every production failure permanently hardens the system.

Section 6: GenAI Math, Case Studies and System Design

25 deeper questions: 10 on the math behind GenAI (attention, the sqrt(d_k) scaling, softmax temperature, cross-entropy and perplexity, KV-cache and LoRA sizing, scaling laws, cosine similarity, RoPE), 5 open-ended case studies that mirror real production incidents, and 10 full system-design case studies.

GenAI Math (10)

Q1. MEDIUM GOOGLE

Write the scaled dot-product attention formula and name each term.

Answer

Attention(Q, K, V) = softmax(Q K^T / sqrt(d_k)) V. Q is the query matrix (n by d_k), K the key matrix (m by d_k), V the value matrix (m by d_v). Q K^T is an n by m score matrix of query-key dot products, softmax turns each row into a probability distribution over the m keys, and multiplying by V produces a weighted average of value vectors. Every output row is a convex combination of value vectors, weighted by how much its query matches each key.

Q2. HARD OPENAI

Why do we divide by sqrt(d_k) in the attention score, and what breaks if we do not?

Answer

If query and key components are independent with zero mean and unit variance, their dot product over d_k dimensions has variance d_k, so its magnitude grows like sqrt(d_k). Feeding large-magnitude scores into softmax pushes it into a saturated regime where one entry is near 1 and the rest near 0. In that regime the softmax gradient is nearly zero, so learning stalls. Dividing by sqrt(d_k) rescales the scores back to roughly unit variance, keeping softmax in a responsive region with healthy gradients. Without it, deep or wide-head models train slowly or not at all.

Q3. MEDIUM META

Derive the memory and compute cost of self-attention in sequence length n, and explain why long context is expensive.

Answer

The score matrix Q K^T is n by n, so it takes O(n^2) memory and O(n^2 d) compute to form and apply, where d is the head dimension. Doubling context quadruples both. At 128k tokens the matrix has about 16 billion entries per head per layer, which is why naive attention is the bottleneck for long context. FlashAttention avoids materializing the full matrix by tiling in SRAM, cutting memory to O(n) while keeping the same asymptotic compute.

Q4. MEDIUM ANTHROPIC

Write the softmax with temperature and explain the effect of T on the distribution.

Answer

softmax(z_i / T) = exp(z_i / T) / sum_j exp(z_j / T). As T approaches 0 the distribution sharpens toward a one-hot on the largest logit, giving greedy, deterministic output. As T grows the distribution flattens toward uniform, giving more diverse and random output. T = 1 is the unscaled softmax. Temperature is a monotonic reshaping of the same ranking, so it changes how confidently you sample, not which token is most likely.

Q5. MEDIUM GOOGLE

State the cross-entropy loss used for next-token prediction and its relationship to perplexity.

Answer

For a target token with true distribution as a one-hot on token t, the per-token loss is minus log p_theta(t | context). Averaged over a corpus of N tokens the loss is L = -(1/N) sum log p_theta(x_i | x_<i). Perplexity is exp(L), the exponentiated average negative log-likelihood. Perplexity of P means the model is on average as uncertain as a uniform choice among P tokens, so lower is better and it is the natural scale for comparing language models.

Q6. HARD META

Explain the KV-cache memory formula and why it dominates long-context serving.

Answer

Per token the cache stores a key and value vector for every layer and every KV head: size = 2 (K and V) times L layers times h_kv heads times d_head times bytes-per-element. Total cache = that times sequence length times batch size. It grows linearly in both context length and batch, and it is read on every decode step, so for long context it dominates both VRAM and memory bandwidth. This is why GQA and MQA, which cut h_kv, and KV quantization, which cuts bytes-per-element, matter so much for serving.

Q7. MEDIUM NVIDIA

Give the parameter count of a LoRA adapter and compare it to full fine-tuning.

Answer

For a weight matrix W of shape d by k, LoRA learns W plus B A where B is d by r and A is r by k, with rank r much smaller than d and k. Trainable parameters drop from d times k to r times (d plus k). For d = k = 4096 and r = 8 that is about 65 thousand parameters versus about 16.8 million, roughly a 250x reduction, which is why many adapters fit in memory and can be swapped or merged at serving time.

Q8. HARD OPENAI

State the Chinchilla compute-optimal scaling result and what it implies for a fixed budget.

Answer

For a fixed training compute budget C measured in FLOPs, roughly C is about 6 N D where N is parameters and D is training tokens. The compute-optimal allocation grows N and D together, with the Chinchilla finding that tokens should scale about in proportion to parameters, near 20 tokens per parameter. The implication is that many early large models were undertrained: for the same budget a smaller model trained on more data generalizes better. It also motivates training smaller models on more tokens when inference cost matters.

Q9. MEDIUM DATABRICKS

Define cosine similarity and explain why it is preferred over Euclidean distance for embeddings.

Answer

cos(u, v) = (u dot v) / (norm(u) times norm(v)), the dot product of the unit-normalized vectors, ranging from -1 to 1. It measures the angle between vectors and ignores their magnitude, so two texts with the same meaning but different vector lengths still score as similar. Embedding models are typically trained so that semantic similarity corresponds to angle, and cosine is scale-invariant, which makes it more robust than raw Euclidean distance for retrieval.

Q10. HARD META

Walk through the RoPE rotation and why it encodes relative position.

Answer

Rotary Position Embedding splits each query and key into 2D pairs and rotates pair i at position m by angle m times theta_i, where theta_i = 10000^(-2i/d). Because a rotation by m on the query and by n on the key leaves their dot product depending only on the difference m minus n, the attention score becomes a function of relative position rather than absolute position. This gives cleaner relative-position behaviour and better length extrapolation than adding fixed absolute position vectors.

Case Studies (5)

Q11. HARD ATLASSIAN

Case study: your RAG assistant in Confluence started returning outdated answers this week even though nothing shipped. Diagnose and fix.

Answer

First separate retrieval from generation using traces: check whether the retrieved chunk IDs point to stale documents. Since no code shipped, the likely causes are a stale or partially failed re-index, deleted or moved source pages that the index still holds, or a silent embedding-model change on the provider side. Confirm with retrieval-score and stale-document monitors and by diffing index timestamps against source edit times. Fix by repairing the ingestion pipeline (incremental re-index on document change, tombstones for deletions), pinning the embedding model version and re-embedding if it changed, adding a freshness field to rank recent content, and adding a monitor that alerts when index age exceeds source age. Finally add the failing queries to the eval set as regression cases.

Q12. HARD STRIPE

Case study: an agent that issues refunds occasionally refunds the wrong customer. How do you find the root cause and prevent recurrence?

Answer

Pull the traces for the bad runs and inspect the tool-call spans: the failure is almost always wrong arguments (a hallucinated or swapped customer or order id) rather than the wrong tool. Confirm by checking argument-correctness against the resolved entity. Root causes to rule out: ambiguous entity resolution in the prompt, context bleed between concurrent sessions, and indirect injection from ticket text. Prevent recurrence with strict argument validation and schema checks before execution, an idempotency key and a confirmation or human-approval gate for money-moving actions, per-session state isolation, treating ticket and tool content as untrusted, and a tool-use eval suite that scores argument correctness. Add every confirmed bad case to the regression set and monitor refund-tool argument-validation failures.

Q13. HARD META

Case study: after a model upgrade your support bot's cost per conversation jumped 40 percent with no quality gain. Investigate.

Answer

Break cost into input tokens, output tokens, cached tokens, and reasoning tokens per conversation from the spans, and compare old versus new model. Common causes: the new model emits longer or more verbose answers, uses more reasoning tokens, is more retry-happy on tool calls, or lost cache hits because the prompt format changed and broke prefix caching. Also check whether the agent now takes more steps per conversation. Fix by tightening max-output caps and system-prompt length, restoring stable prompt prefixes to regain caching, tuning the model to a cheaper tier for simple intents via a router, capping steps and reasoning effort, and gating the upgrade on a cost budget in CI. Re-run the eval suite to confirm quality is unchanged so you can justify reverting or routing.

Q14. HARD NETFLIX

Case study: p99 latency for your chat feature is 6x the p50 during peak hours, but off-peak it is fine. What is happening and how do you fix it?

Answer

The pattern points to load-driven contention rather than a code bug. Use traces to split latency into admission or queueing, prefill (TTFT), and decode. At peak, likely causes are KV-cache pressure forcing smaller effective batches or evictions, cold-start scaling that cannot keep up with the ramp, head-of-line blocking where long prefills stall short interactive requests, and downstream tool or retrieval timeouts under load. Fixes: pre-scale with warm pools ahead of the daily peak, add admission control and priority queues so interactive traffic is not blocked by batch or long prefills, disaggregate prefill and decode pools, cap max sequence length, add KV quantization to fit more sequences, and set autoscaling on tokens-in-flight rather than CPU. Validate with a peak-like load test.

Q15. HARD ANTHROPIC

Case study: your eval scores look great but users complain the assistant is unhelpful in production. Reconcile the gap.

Answer

This is the classic offline-online divergence. Likely causes: the golden eval set is stale or unrepresentative of real traffic, data contamination inflated scores, the judge is miscalibrated or biased toward verbosity, or the metrics you optimized (for example faithfulness) do not capture what users actually value (task completion, latency, tone). Reconcile by measuring online signals directly (thumbs, edits, retries, deflection, task success) and correlating them with your offline metrics; where they disagree, trust the user signal. Rebuild the eval set from recent production traces including the failing ones, decontaminate it, recalibrate the judge against fresh human labels, and add the missing dimensions as metrics. Then close the loop so production failures continuously feed the eval set.

System Design Case Studies (10)

Q16. HARD GOOGLE

System design: design a production RAG system for 10 million enterprise documents serving thousands of queries per second.

Answer

Ingestion: pipeline that chunks documents with overlap, embeds them in batch, and writes to a sharded vector DB with per-document access-control metadata and incremental re-indexing on change. Retrieval: hybrid search (dense plus BM25) with reciprocal-rank fusion, then a cross-encoder reranker on the top candidates, filtered by the caller's ACLs. Generation: inject the reranked chunks with citation-enforced prompting behind a model gateway. Serving: vLLM with continuous batching and autoscaling, semantic cache for repeated queries, replicas of the vector index for QPS, and regional co-location. Reliability: rate limits, fallbacks, circuit breakers. Quality: online groundedness evals on sampled traffic, retrieval-score and drift monitors, and a golden eval set gating deploys.

Q17. HARD META

System design: design an observability and evaluation platform for all LLM traffic across a large company.

Answer

Instrument every app once with the OTel GenAI conventions and export via OTLP through a central Collector that applies redaction, sampling, and routing. Store traces, metrics, and logs in a scalable backend keyed by trace_id with tenant, feature, model, and version tags. Layer an async evaluation service that consumes sampled spans and writes quality and safety scores back onto them. Provide dashboards for latency, cost, and quality sliced by any tag; SLO and burn-rate alerting; and a dataset service that promotes interesting or failing traces into versioned golden sets. Add a CI integration so teams gate deploys on those sets. Keep it vendor-neutral so backends and providers can change without re-instrumentation.

Q18. HARD AMAZON

System design: design a multi-tenant LLM gateway that many product teams share.

Answer

A stateless gateway service behind a load balancer exposing a unified API. Core features: authentication and per-tenant API keys, per-tenant token and request rate limits with backpressure, routing and load-balancing across providers and models with health-based failover and fallback chains, a semantic cache scoped per tenant, and circuit breakers. Cross-cutting: cost tracking and budgets per tenant and feature, full OTel tracing with tenant and model tags, prompt-version and model-version stamping, and secrets in a manager with rotation. Add admission control and priority tiers so one tenant cannot starve others, and a config plane so routing rules and model choices change without redeploying client apps.

Q19. HARD STRIPE

System design: design a customer-support agent that can read orders, issue refunds, and escalate to humans.

Answer

Stateless agent runtime with external session and memory stores. Tools exposed via an internal MCP server over Streamable HTTP with OAuth 2.1, audience-bound tokens, and per-tool least-privilege scopes tied to the acting user. Money-moving tools require argument validation, idempotency keys, and a human-approval gate; read tools are lower risk. Guardrails: treat ticket and tool content as untrusted (indirect-injection defense), input and output filters, and PII redaction. Budgets: step, tool, token, and time caps with circuit breakers. Escalation path to a human queue when confidence is low or a gate triggers. Observability: full trajectory tracing, tool-argument-correctness evals, refund-error monitoring, and audit logs for every action.

Q20. HARD NETFLIX

System design: design an LLM serving platform that meets a strict 200ms time-to-first-token SLO at scale.

Answer

Optimize the model path first: quantize, enable KV cache, and run vLLM or TGI with PagedAttention and continuous batching. Disaggregate prefill and decode into separate GPU pools so long prefills do not block short interactive requests, and use priority queues for interactive traffic. Keep warm pools and pre-scale ahead of daily peaks; autoscale on tokens-in-flight and queue depth, not CPU. Put a gateway in front for rate limiting, semantic caching, and fallbacks, and co-locate regionally to cut network latency. Monitor TTFT as its own SLO with burn-rate alerts, split traces into admission, prefill, and decode phases, and load-test at peak concurrency to verify the tail, not just the median.

Q21. HARD MICROSOFT

System design: design an enterprise MCP platform so internal AI agents can safely use hundreds of tools.

Answer

A registry or catalog of approved MCP servers, each deployed as stateless Streamable HTTP behind a gateway on Kubernetes with autoscaling. Security: OAuth 2.1 with PKCE, per-server audience-bound tokens (RFC 8707), a central authorization server, and per-tool least-privilege scopes mapped to user roles. Governance: pin and negotiate mcp.protocol.version, review and sign tools before catalog listing, and treat all tool output as untrusted. Observability: trace every tools/call with W3C context, mcp.session.id, and per-tool latency and error SLOs, plus audit logs. Reliability: rate limits, circuit breakers, blue-green server upgrades, and the Tasks pattern for long-running tools. Provide a self-service onboarding flow with policy checks.

Q22. HARD DATABRICKS

System design: design an offline plus online evaluation pipeline that gates every model and prompt change.

Answer

Offline: a versioned golden dataset built from real production traces, with metrics per layer (retrieval precision and recall, faithfulness and relevancy, tool-argument and trajectory accuracy) run as CI tests with threshold assertions and significance-aware comparisons. Block merges on regressions. Online: sample production traffic and run async evals with a calibrated, version-pinned judge, writing scores onto traces; monitor score distributions with burn-rate alerts. Rollout: shadow, then canary with online eval gates, then progressive rollout with automated rollback. Feedback: promote failing production cases back into the golden set. Guard the judge itself with position-bias tests and periodic human calibration.

Q23. HARD UBER

System design: design a real-time cost-control and budgeting system for autonomous agents.

Answer

Per-request enforcement at the runtime: hard caps on steps, tool calls, tokens, and wall-clock, with interruption on breach and loop detection via repeated-span checks. A budgeting service tracks spend in real time per tenant, feature, and user from token and tool telemetry, with soft and hard thresholds. When a budget nears its limit, degrade gracefully by routing to a cheaper model, tightening caps, or refusing. Emit tokens-per-request and steps-per-request distributions to monitoring with tail-growth alerts. A gateway enforces per-tenant rate limits and admission control so one runaway agent cannot exhaust shared capacity. All limits are configurable per tenant without redeploying.

Q24. HARD ATLASSIAN

System design: design an AI feature embedded across Jira and Confluence that respects per-user document permissions.

Answer

Every retrieval must be permission-aware: store per-document ACLs as vector-DB metadata and filter candidates by the requesting user's permissions before reranking and generation, so the model never sees content the user cannot access. Stateless agent runtime with external session state keyed per workspace. Tools exposed via an internal MCP server scoped to the acting user with least privilege; writes require confirmation and are audit-logged. Guardrails against indirect injection from document content. Observability sliced by workspace and feature, with per-tenant quality and cost dashboards and anomaly detection. Rollout via canary per workspace cohort with eval gates and instant rollback, and a golden set drawn from real cross-permission scenarios.

Q25. HARD OPENAI

System design: design a safe deployment and rollback system for shipping frequent prompt and model changes.

Answer

Treat prompts, tool schemas, model choice, and parameters as one versioned artifact under source control with review. CI runs the eval regression suite plus safety and red-team checks and latency and cost budget gates; nothing ships without passing. Rollout is layered: offline gate, shadow (mirror traffic, compare quality, latency, and cost with no user impact), canary with online eval gates and metric-based ramp, then progressive rollout by region or tenant. Every request trace is stamped with the artifact version so regressions are attributable. Automated rollback triggers on SLO, quality, or cost burn, and blue-green keeps an instant revert path. Confirmed production failures feed back into the eval set to harden future releases.

Quick Reference: What Each Section Drills

Theme	Section	Signature Topics
Observability	1	OTel gen_ai.* spans, token/cost/latency, RAG telemetry, execution graphs
Traceability	2	trace/span IDs, W3C context, MCP tool traces, span links, replay
Monitoring	3	SLOs, drift, hallucination rate, guardrails, cost anomalies, alerting
Evals	4	offline/online, golden sets, LLM-judge bias, RAGAS, trajectory eval
Serving Infra	5A	vLLM, PagedAttention, continuous batching, GPU autoscale, quantization
MCP Deployment	5B	stdio vs Streamable HTTP, OAuth 2.1 + PKCE, stateless scaling, Tasks
Agent Deployment	5C	stateless runtime, step/cost budgets, injection defense, multi-agent
Gateways and CI/CD	5D	model gateway, fallbacks and circuit breakers, PromptOps, canary/shadow
GenAI Math	6	attention, sqrt(d_k), softmax temp, perplexity, KV cache, LoRA, RoPE
Case Studies	6	RAG staleness, agent misfire, cost spike, latency tail, offline-online gap
System Design	6	RAG at scale, gateways, agents, serving SLOs, MCP platform, eval pipeline